Algebra: 5-minute pitch script

Source: docs/pitch/SCRIPT.md

Algebra: 5-minute pitch (2 min slides · 3 min live demo · close)

Deck: `Algebra-pitch.pptx` (7 slides, the same script is in each slide's speaker notes). Rebuild after edits: `python build_deck.py Algebra-pitch.pptx` (needs `pip install python-pptx`).

Time	What's on screen
0:00 to 2:00	Slides 1 to 5
2:00 to 2:10	Slide 6 "Watch it say no", switch to the browser
2:10 to 4:40	Live demo: the router and the spend firewall
4:40 to 5:00	Slide 7, close

Part 1: slides (2 min, about 270 words)

1 · Title (15 s)

Hi, we're building Algebra. AI agents are starting to pay for things themselves: data, search, inference, all paid per call in USDC over x402. Algebra is the router and the spend firewall that sits between the agent and that money.

2 · The problem (25 s)

The moment an agent can pay, three questions come up, and none of them can be left to the model. One: which provider? There are over 1,600 paid APIs across four catalogs. Some are down, some charge more than they list, and some are honeypots priced at $25 a call, waiting for an agent that pays whatever it's asked. Two: how do you stop it overspending? A line in the prompt is not a budget. Three: did it pay exactly once? A timeout doesn't tell you whether money moved.

3 · How it works (30 s)

Here's how it works. The agent doesn't pick a provider. It asks for a kind of work, say a token price, and the most it will pay. Its Spend Pass checks the request first: budget, per-call cap, allowed providers, and a kill switch. Then the router asks every provider that can do the job for its real price, all at once. It drops the ones that are down, the ones that overcharge, and the traps. It pays the best one in USDC on Solana, from a wallet the agent never holds, and hands back the answer with a signed receipt. It's one REST call or an MCP tool.

4 · Why Solana (25 s)

And this is why it's on Solana. The Spend Pass is an account owned by our own Anchor program, live on devnet. Your wallet funds a vault. Algebra can pull one payment at a time, and the program checks every pull against the cap, budget, window and expiry. You can freeze it from your own wallet and we can't undo that. Even a compromised Algebra server couldn't take more than the pass allows.

5 · Where it stands (20 s)

Over 1,600 providers from four catalogs, grouped into 14 kinds of work. Real devnet USDC payments, including pulls from the on-chain pass. The same request sent twice pays once. Mainnet is read-only on purpose until the program is audited. Now let me show you the core of it.

6 · Demo divider (5 s, while switching windows)

I'll do this in the sandbox so nothing depends on a faucet, but it's the same code path that pays real USDC on devnet.

Part 2: live demo (3 min), the router and the spend firewall

The one idea to land: *the agent asks for work and a ceiling, and Algebra refuses the bad providers for you, pays the good one once, and stops everything at one click.*

This runs in the **sandbox**: simulated USDC and simulated sellers, some built to misbehave. No wallet, no faucet, no LLM key, so very little can break on stage.

Before you go on stage

- [ ] Docker/OrbStack running, then `ALGEBRA_MASTER_KEY=x docker compose -f deploy/docker-compose.yml up -d postgres redis`
- [ ] API: `cd backend && set -a && source .env && set +a && go run ./cmd/api` → `curl localhost:8080/readyz` shows `{"postgres":"ok","redis":"ok"}`
- [ ] Web: `cd frontend && pnpm dev` → http://localhost:3000
- [ ] **Sign up a fresh account in advance** (signup takes time on stage), and leave it on **Console** → **Quickstart** with *no pass yet*.
- [ ] Open a second tab on **Console** → **Firewall** (`/console/firewall`).
- [ ] Browser zoom at 125 to 150% so the back row can read it. Close other tabs and turn off notifications.
- [ ] Do one full rehearsal on a *different* account, so the stage account is still empty.

The run (Quickstart page: `/console/quickstart`)

0:00 · Create the policy (20 s). Click **Create a sandbox pass (1 USDC, 0.05 a call)**.

This is the Spend Pass, the agent's limits: one dollar a month, five cents a call at most. The agent never gets a wallet, only this.

(Skip step 3, the API key, to save time. If you have spare seconds: "a key says who's calling, a pass says what can be spent. Two separate authorities.")

0:20 · Ask for work, not a provider (15 s). Leave **Token price** and **Cheapest** selected.

The agent asks for "token price", not a provider, with a ceiling of five cents.

0:35 · Dry run: the money shot (60 s). Click **Dry run**. The result shows `ALLOW · would pay sandbox:beta $0.001`, then the refusals. Point at each one:

Before spending anything, Algebra priced every provider that can do this. Look at who it refused: - **flaky**: *down*. It failed its last three health probes. - **greedy**: lists a tenth of a cent but actually asks for four tenths. *Price above listing*, refused. - **trap**: twenty-five dollars for one call, 8,333 times the usual price. Refused. - **shifty**: this one's sneaky. It suddenly asked to be paid at a *different address* than the one we pinned. That's how you'd steal from an agent. Blocked. - And everything below that is real mainnet providers from the catalogs, Birdeye, Alchemy, CoinGecko, held back because this pass is sandbox-only.

So it would pay beta, a tenth of a cent, with alpha as the fallback. And nothing has been paid yet. That was a dry run.

1:35 · Execute (35 s). Click **Execute in the sandbox**.

Now for real. *"Committed: paid once and the result was received."* Beta, a tenth of a cent. Here's the provider's answer, and here's a signed Ed25519 receipt. Anyone can verify it against our public key, and we can't forge or change it after the fact.

(Optional, if ahead of time: click Verify it under the receipt to show it verifying.)

2:10 · Kill switch (40 s). Switch to the **Firewall** tab and click **Freeze every pass**. Back to Quickstart, click **Execute in the sandbox** again.

And if something goes wrong, say the agent starts looping or behaving strangely, one click freezes every agent. Watch… refused: *"frozen by its owner's kill switch."* Nothing paid. It's checked again right before a payment is signed, so it stops even a payment in flight. And with the on-chain pass, you can do this from your own wallet.

Go back to Firewall and click **Lift the kill switch** (keep talking while you do).

2:50 · Wrap (10 s).

So that's the core: the agent asks for an outcome and a ceiling, Algebra picks the provider, refuses the bad ones, pays once, and you keep the off switch.

Switch back to the deck, slide 7.

If something breaks

Problem	Fix, without stopping the talk
Dry run spins or errors	Click it once more. If it still fails, narrate slide 3's code block: it shows exactly that result from a real devnet run.
The kill switch error looks different	The point still lands: the payment was refused. Read whatever error appears.
"Not signed in"	Have a second pre-made account logged in on another browser profile.
Whole app down	Go to slide 3 and slide 5, and say: "these are recorded devnet transactions, links in docs/DEMO.md".

Part 3: close (20 s), slide 7

So that's Algebra: agents can spend, and Algebra decides how much. Next is an audit of the program, then real mainnet payments behind on-chain passes. Thank you, happy to take questions.

Likely questions, short answers

- **Why not just give the agent a wallet with a small balance?** A balance limits the total, not *who* gets paid, *how much per call*, or *how fast*. It also does nothing about overpriced or fake providers. And you can't take a balance back mid-payment.
- **Who are the devnet providers?** Our own demo providers. The public catalogs only list mainnet endpoints. The payments, the program and the settlement are real devnet transactions.
- **Is it on mainnet?** Read-only today: discovery, quotes and dry runs. Payments are switched off until the program is audited and every mainnet payment has to come from an on-chain pass.
- **How does it make money?** Plans for teams (defined, not yet billing) on top of the execution layer.
- **What stops Algebra itself from stealing?** With an on-chain pass the program enforces the limits. The server can pull only within the per-call cap and budget, only into its payer, and never after the owner freezes it.